

Informe de Revisión de Seguridad Generador de Ramales

Revisión enfocada de seguridad sobre el conjunto del código del proyecto —pipeline geoespacial, interfaz web e ingesta de datos— orientada a identificar vulnerabilidades explotables con impacto real. No sustituye a una auditoría formal exhaustiva.

Alcance: proyecto completo · Fecha: 2026-07-14 · Método: /security-review

01 Veredicto

Sin vulnerabilidades de seguridad explotables.

La revisión de todo el código del proyecto no identificó ninguna vulnerabilidad explotable (Alta 0 · Media 0 · Baja 0). Se recogen dos observaciones de robustez de defensa en profundidad (sección 04) que **no** constituyen vulnerabilidades.

02 Alcance y metodología

CÓDIGO REVISADO

Todo el código fuente bajo `proyecto/src/` (37 módulos Python), los ficheros de configuración (`escenario.yaml`, `escenario_base.yaml`, `perfiles.yaml`), la interfaz `streamlit_app.py`, el generador de informes, la CLI y los scripts de entorno. Se organizó en tres bloques: **ingesta y red** (descargas WFS/WMS/REST, Catastro, OpenTopography, IGME; descompresión de archivos), **aplicación** (Streamlit, CLI, informes) y **cómputo** (superficies de coste, motor de trazado, métricas y comparación).

MODELO DE AMENAZA

Se consideran no confiables: (a) las respuestas y archivos devueltos por servidores GIS/geocodificación externos y (b) los valores de la configuración YAML. Las variables de entorno y los argumentos de línea de comandos se consideran de confianza. Para cada patrón de riesgo se trazó el flujo de datos desde la fuente no confiable hasta el punto sensible, reportando solo hallazgos con alta confianza de explotación.

EXCLUSIONES

Por diseño del propio comando se excluyen denegación de servicio, secretos almacenados en disco, límites de tasa, dependencias de terceros desactualizadas y recomendaciones de mero endurecimiento.

03 Categorías examinadas

CATEGORÍA	RESULTADO
Path traversal / ZIP slip. Descompresión de archivos descargados de Catastro/INSPIRE (convertir_catastro.py, alinear_capas.py).	Sin hallazgos
Deserialización YAML. Carga de configuración en ingesta, superficie y trazados.	Sin hallazgos
Inyección de comandos. Uso de subprocess en la orquestación de escenarios.	Sin hallazgos
SSRF (host/protocolo). Llamadas requests/urlopen a servicios externos.	Sin hallazgos
XSS / inyección de HTML. Render con unsafe_allow_html en Streamlit e informes.	Sin hallazgos
Escritura con nombres no confiables. Ficheros de salida a partir de datos externos.	Sin hallazgos
Ejecución de código / deserialización insegura. eval/exec/pickle.	Sin hallazgos
Verificación TLS. Certificados en las descargas HTTPS.	Sin hallazgos

Puntos verificados de forma destacada:

- **ZIP slip neutralizado.** Las rutas de descompresión usan `ZipFile.extractall/extract` (o nombres reducidos a `Path( ... ).name`), que desde CPython 3.6.2 sanean los nombres de entrada (eliminan unidades, separadores iniciales y componentes `..`), por lo que una entrada maliciosa `../../evil` de un archivo remoto se colapsa a un nombre seguro.
- **YAML siempre seguro.** Todas las cargas emplean `yaml.safe_load`; no existe ningún `yaml.load` con cargador inseguro.
- **Sin shell.** El único `subprocess.run` (`preparar_escenario.py`) usa una lista de argumentos con `sys.executable` y nombres de módulo fijos, sin `shell=True`.
- **SSRF fuera de alcance explorable.** Todos los *endpoints* son constantes; la configuración solo aporta coordenadas numéricas que llegan como parámetros de consulta (control de *path*, excluido). Los valores se fuerzan a `float/geometría`, de modo que un valor no numérico falla en vez de inyectar.
- **XSS descartado.** Los `unsafe_allow_html` con datos variables interpolan constantes de la app o números calculados; el único texto libre del usuario (nombre de escenario) se sanea con `re.sub(r"[^A-Za-z0-9_-]", "")`, y las coordenadas son entradas numéricas.
- **TLS activo.** No hay `verify=False`; la reubicación del *bundle* de certificados a una ruta ASCII mantiene la verificación de certificados.

04 Observaciones de robustez (no vulnerabilidades)

Ninguna de estas observaciones es explotable en el estado actual del código; se anotan como mejoras de defensa en profundidad.

1 · Escapado defensivo en una rama de HTML actualmente inalcanzable

La función `_lista_municipios_html` (`streamlit_app.py`) interpola el nombre y la URL devueltos por el feed ATOM de Catastro en HTML sin `html.escape`. Hoy su único llamador se ejecuta solo cuando la lista de municipios está *vacía*, por lo que ese dato nunca llega al HTML en tiempo de ejecución. Recomendación: aplicar `html.escape()` para que la función siga siendo segura si un futuro llamador le pasa una lista no vacía.

2 · Saneado del identificador de perfil usado en nombres de salida

El `id` de `perfiles.yaml` se emplea sin sanear al construir nombres de fichero de salida (p. ej. `ruta_{s}_{pid}.gpkg`). No es un hallazgo confirmado porque requiere que el atacante ya controle la configuración —que es el propio límite de confianza— y el impacto se limita a escribir un fichero válido en una ruta elegida. Recomendación: validar el `id` frente a un patrón `[A-Za-z0-9_-]+` como refuerzo.

05 Conclusión

El proyecto muestra un manejo de entradas limpio: carga YAML segura, descompresión de archivos sin traversal, subprocessos sin shell, verificación TLS activa y ausencia de *sinks* de ejecución de código o de HTML alcanzables con datos no confiables. **Resultado: 0 hallazgos** (Alta 0 · Media 0 · Baja 0), con dos recomendaciones opcionales de robustez.